

Query Evaluation and Optimization

Internals of a DBS I

Query Optimization
And Execution

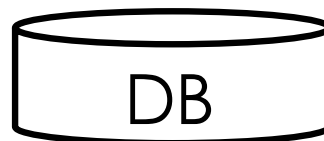
Relational Operators

Files and
Access Methods

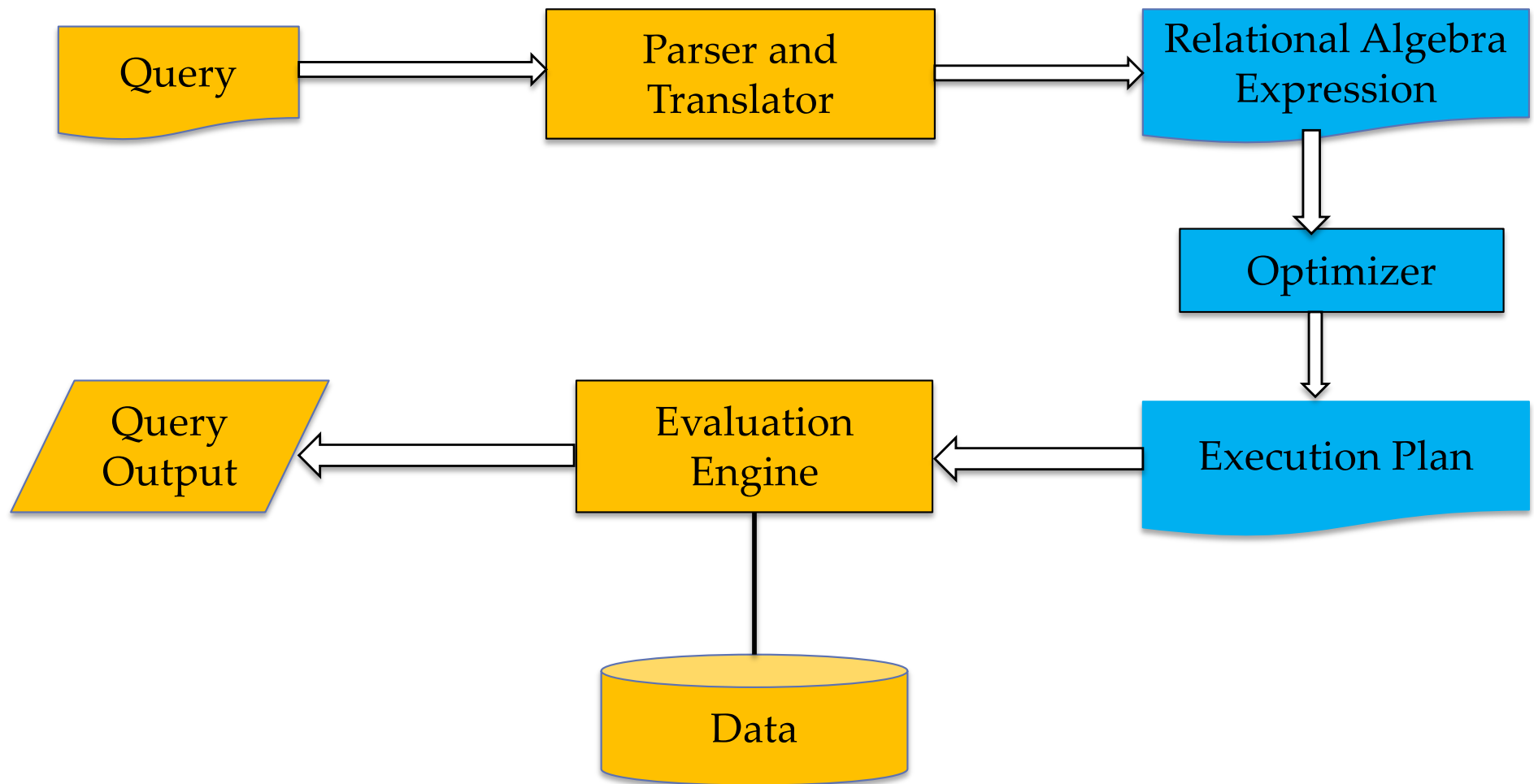
Buffer Management

Disk Space
Management

|

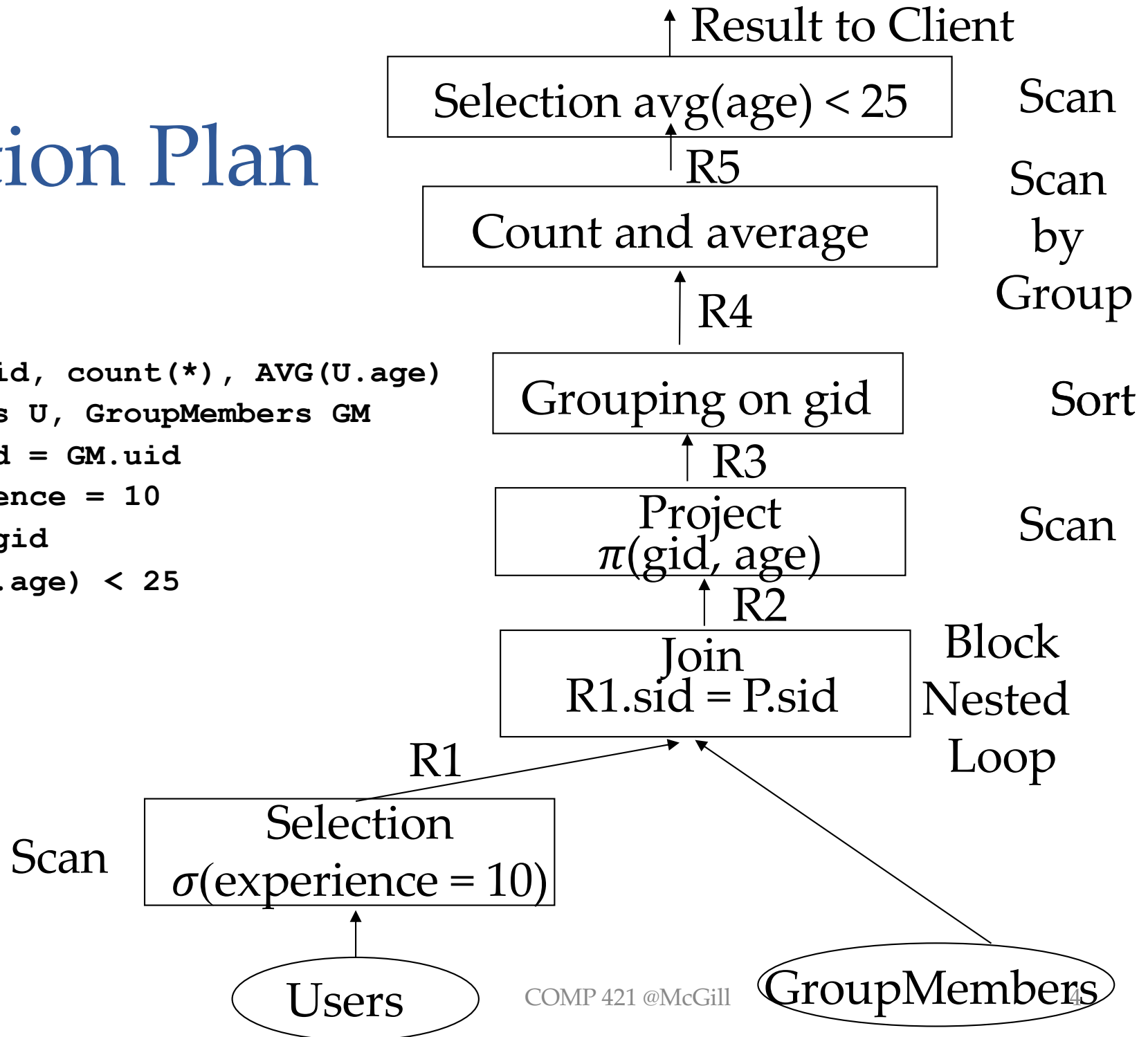


From Query to Result



Execution Plan

```
SELECT  GM.gid, count(*), AVG(U.age)
FROM    Users U, GroupMembers GM
WHERE   U.uid = GM.uid
AND U.experience = 10
GROUP BY GM.gid
HAVING AVG(U.age) < 25
```



From basic operations to execution plan of a complex query

- Execution tree:
 - Leaf nodes are the base relations
 - Inner nodes are operators
 - For each operator, an indication how it will be executed (index nested loop, sort, index, simple scan....)
 - Tuples from base relations (leaves) flow into the parent operator node(s)
 - Output of an operator node flows into the parent node
 - Output of the root flows as result to the client

Example: Step by step

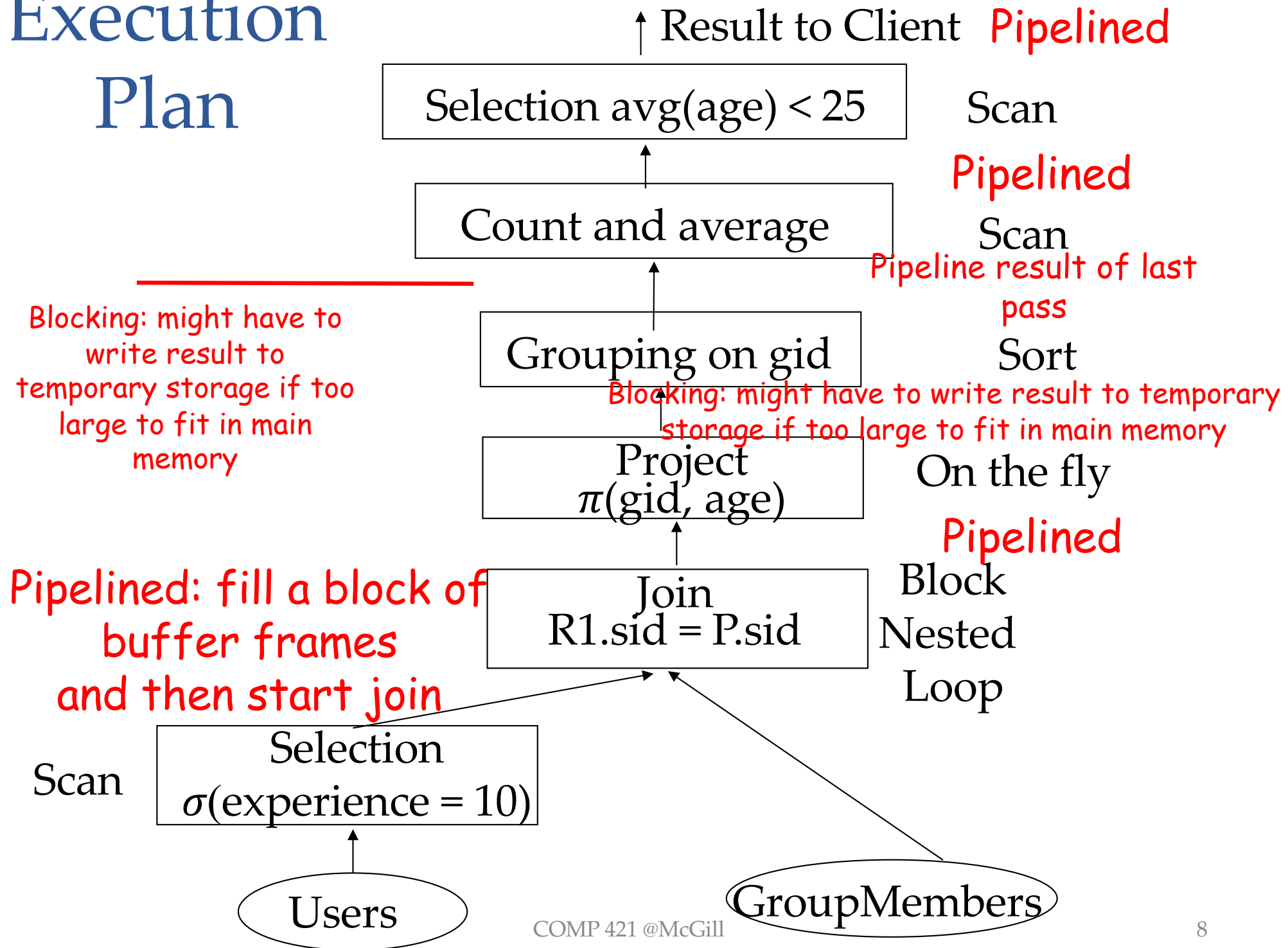
```
SELECT  GM.gid, count(*) , AVG(U.age)
FROM    Users U, GroupMembers GM
WHERE   U.uid = GM.uid AND U.experience = 10
GROUP BY GM.gid
HAVING  AVG(U.age) < 25
```

- Scan the Users table, determine all tuples with experience = 10 (scan or index)
 - Results in intermediate relation R1
- Join R1 and GroupMembers (several join options)
 - Results in intermediate relation R2
- Eliminate attributes other than gid and age.
 - Results in intermediate relation R3
- Group the tuples of R3 on gid (done by sort).
 - Results in intermediate relation R4
- Scan R4, count and perform average
 - Results in intermediate relation R5
- Return all tuples with $\text{avg}(\text{age}) < 25$.

Pipelining

- Execution within one operator:
 - As soon as a tuple is determined it is forwarded to next operator for processing **or**
 - As soon as one output frame is full, it is processed by the next operator
- Parallel execution of operators
- No materialization of intermediate relations if it can be avoided
- Iterator operator uses pipelining
 - Every operator is an iterator
 - Iterator provides the following interface to “parent” operators: **open, getNext, close**
 - Iterator calls interface methods of “children” operators

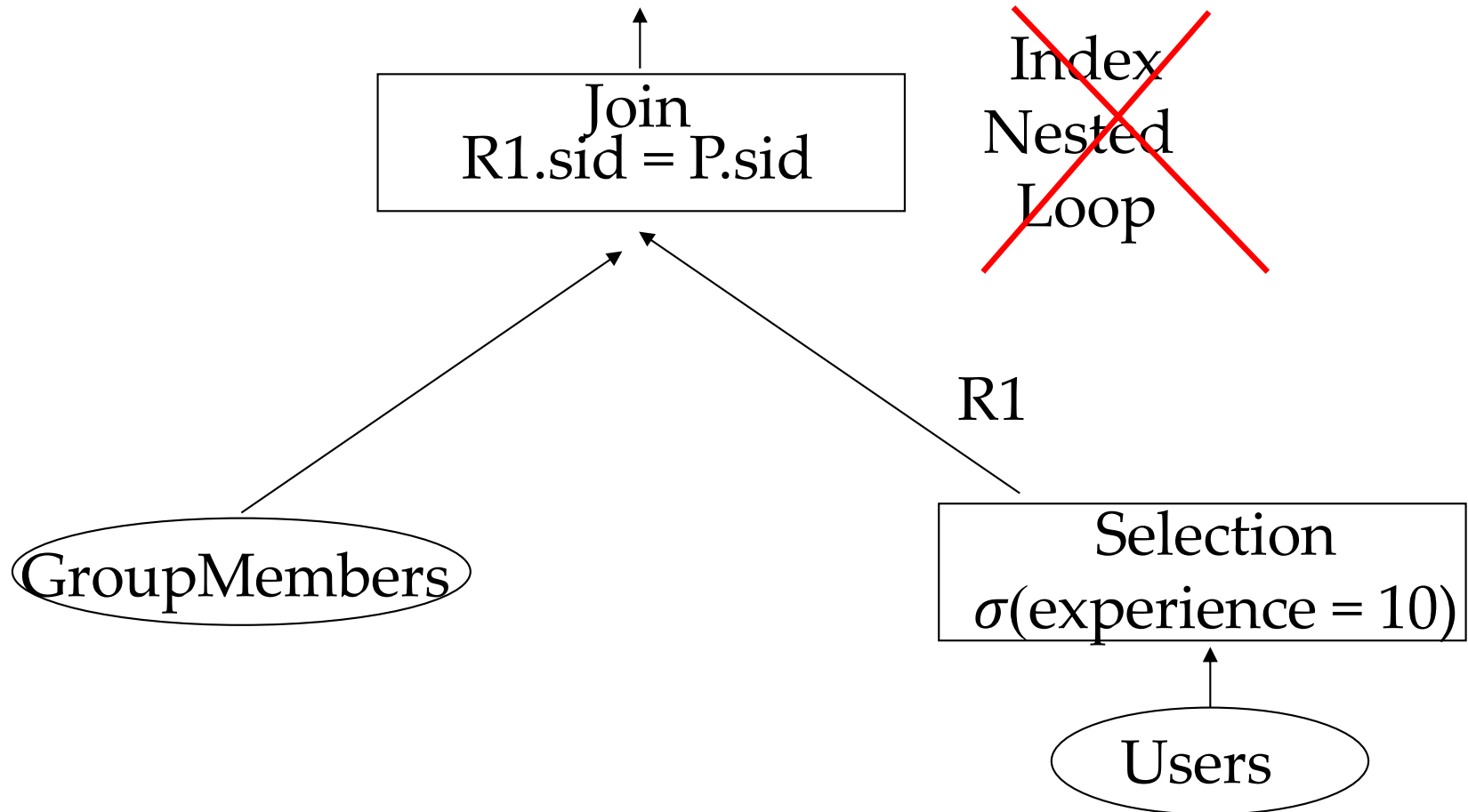
Execution Plan



Equality Join with further restrictions

```
SELECT  *
FROM    Users U, GroupMembers GM
WHERE   U.uid = GM.uid
AND     experience = 10
```

- 50 buffer frames available, 40,000 tuples on 500 UserPages, 1000 GroupMember pages
- assume 1% have experience = 10, i.e., 400 tuples
- Do selection before join (400 tuples fit on at most 5 pages); Stream (pipeline) qualifying tuples into join
- Which join is the best?
 - Block nested loop (get qualifying tuples until block is full)
 - $\text{UserPages} + \text{UserPages} \times 0.01 / 50 \times \text{GroupMemberPages} = 500 + 1 \times 1000 = 1,500$
 - Index nested loop
 - Index on uid of GroupMembers (User outer, GroupMembers inner)
 - $\text{UserPages} + \text{Card}(\text{Users}) \times 0.01 \times \text{Cost finding GroupMembers} = 500 + 400 \times (1 + 2.5) = 500 + 1400 = 1900$
 - Index on uid of User
 - You cannot do a selection on an attribute of User first and then have User as the inner of an index-nested loop join



Equality Join with further restrictions

```
SELECT  *  
FROM    Users U, GroupMembers GM  
WHERE   U.uid = GM.uid  
AND     experience = 10
```

- 50 buffer frames available, 40,000 tuples on 500 UserPages, 1000 GroupMember pages
- assume 1% have experience = 10, i.e., 400 tuples
- Do selection before join (400 tuples fit on at most 5 pages); Stream (pipeline) qualifying tuples into join
- Which join is the best?
 - Sort-Merge Join
 - Pipelining can only be done at last pass of sort; therefore intermediate half-sorted relation (runs)
 - But in particular case: result of selection fits into main memory (1% of UserPages = 5 pages – even less if compact storage); therefore sort of user tuples in main memory
 - Cost: UserPages + 3* GroupPages = 3500

Equality Join with further restrictions

```
SELECT  *
FROM    Users U, GroupMembers GM
WHERE   U.uid = GM.uid
AND     experience = 10
```

- **assume 0.5% have experience = 10** $\rightarrow$ 200 tuples
 - Do selection before join; Stream (pipeline) qualifying tuples into join
 - Block nested loop cost remain the same
 - $\text{UserPages} + \text{UserPages} * 0.005 / \text{blocknumber} * \text{GroupMemberPages} = 500 + 1 * 1000 = 1,500$
 - Index nested loop
 - Index on uid of GroupMembers (User outer, GroupMembers inner)
 - $\text{UserPages} + \text{Card}(\text{Users}) * 0.01 * \text{Cost finding GroupMembers} = 500 + 200 * (1 + 2.5) = 500 + 700 = 1200$
- Assume 50% have experience = 10
 - Block-nested: $500 + 250 / 50 * \text{GroupMemberPages} = 500 + 5 * 1000$
 - Index nested loop: $500 + 20000 * (1 + 2.5)$
 - Merge-Sort
 - Do merge-sort join with selection on the fly
 - Read in the pages of each relation one-by-one, check only users with experience = 10 for the join
 - Cost: $500 + 3000$

Optimization Techniques

- Two Types of optimization
 - Algebraic optimization:
 - Use simple rules to perform those operations first that eliminate a lot of tuples
 - Push down selections and projections
 - Build a basic operator tree
 - Do not yet consider HOW to execute each operator
 - Consider the number of tuples that flow from one operator to the next and their sizes
 - Key issues: statistics
 - Cost-based optimizations
 - Execute operation in an order such that most appropriate access paths (indexes) can be used

Algebraic Optimization: Projection, Selection and Join

❑ Pushing down selections and projections to the relations to which selection refers

❑ Careful with project

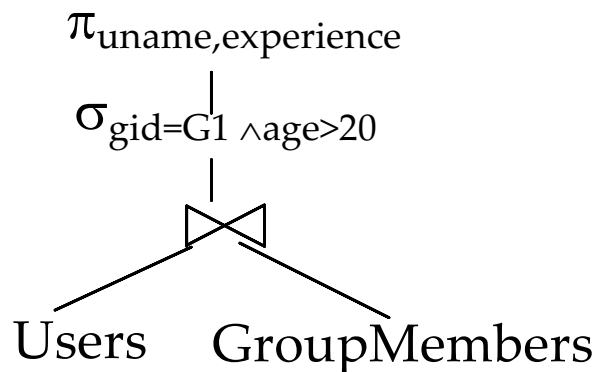
- $\pi_{\text{uname}, \text{experience}}(\sigma_{\text{gid}=\text{G1} \wedge \text{age} > 20}(\text{Users} \bowtie \text{GroupMembers}))$
 - first join, then selection, then project

```
SELECT u.uname, u.experience
FROM   Users u, GroupMembers g
WHERE  u.uid = g.uid
AND    g.gid = 'G1'
AND    u.age > 20
```

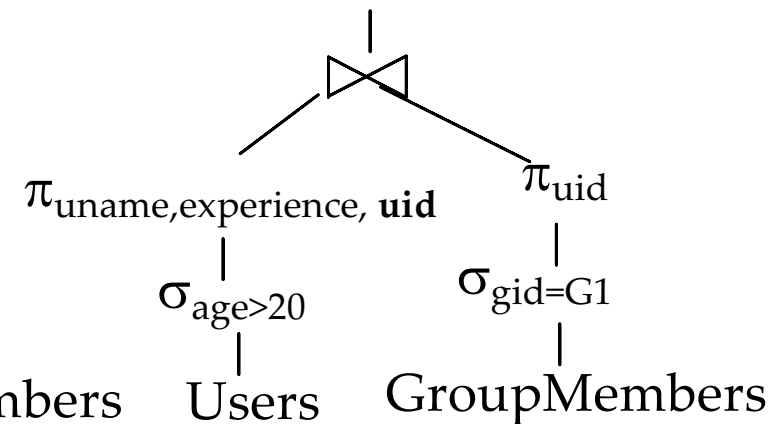
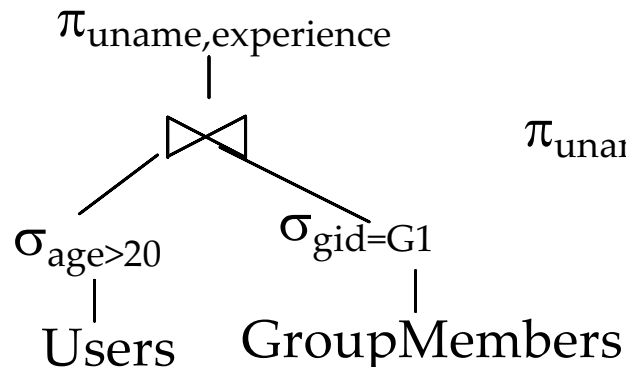
*Push selections AND
projections*

$\pi_{\text{uname}, \text{experience}}$

Not optimized:



Push selections



Push Projections

- Pushing down projections will not reduce the number of tuples but the SIZE of the intermediate results
- Be careful not to lose attributes that you need later on
- $\pi_{\text{uname}}(\text{Users} \bowtie \text{GroupMembers}) \neq \pi_{\text{uname}}(\text{Users}) \bowtie \text{GroupMembers}$

Algebraic Optimization (contd)

```
SELECT S.sname, P.cid
```

```
FROM Skaters S, Participates P, Competitions C
```

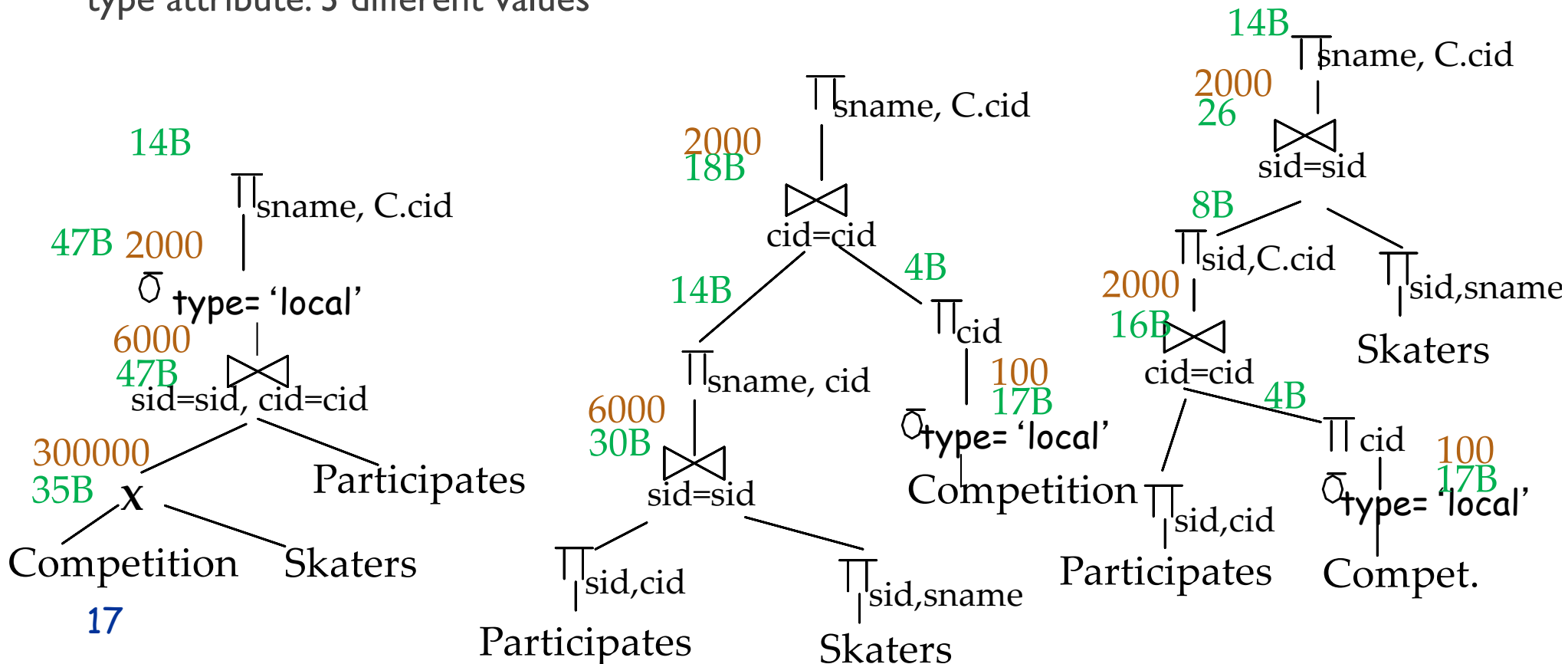
```
WHERE P.sid=S.sid AND P.cid=C.cid AND C.type= 'local' ;
```

Skaters (sid: 4 Byte, sname (10 Byte), age (4Byte)): 1000 tuples

Participates (sid: 4 Byte, cid: 4 Byte, rank (4 Byte): 6000 tuples

Competition (cid: 4 Byte, date (8 Byte), type (5 Byte): 300 tuples

type attribute: 3 different values



Algebraic Optimization

- Push down selections
- Push down projections
 - Project on only those attributes that you need for the operators to come
- Order joins as to have intermediate results
 - With the least number of tuples
 - The smallest tuple sizes

Cost Based Optimization

- Find a plan with low cost
 - Consider possible implementations of operators and calculate step-by-step the I/O costs
- Real Systems: Dynamic programming in bottom-up fashion for deep join plans :
 - Pass 1: Find best 1-relation plan for each relation.
 - Pass 2: Find best way to join result of each 1-relation plan to another relation. (*All 2-relation plans.*)
 - Consider inner and outer
 - Pass N: Find best way to join result of a (N-1)-relation plan (as outer) to the N' th relation. (*All N-relation plans.*)
 - Prune high-costs alternatives
 - Retain alternatives with interesting features
 - Cheapest plan overall, plus
 - Cheapest plan for each *interesting order* of the tuples.
 - Bottom-up calculation each possible execution plan, estimate the cost

This course: hybrid algebraic and cost-based

Example

```
SELECT u.uname, u.experience
FROM   Users u, GroupMembers g
WHERE  u.uid = g.uid
AND    g.gid = 'G1'
AND    u.age > 35
```

Users (U):

40,000 tuples (denoted as CARD(U))

Around 80 tuples per data page

500 data pages (denoted as UserPages)

A non-clustered index on uid has around 170 leaf pages

A non-clustered index on age

Age min:11, age max: 60

GroupMembers (GM):

100,000 tuples (denoted as CARD(GM))

Around 100 tuples per data page

Total of 1000 data pages (denoted as GroupMemberPages)

Non-clustered index on gid (500 different groups)

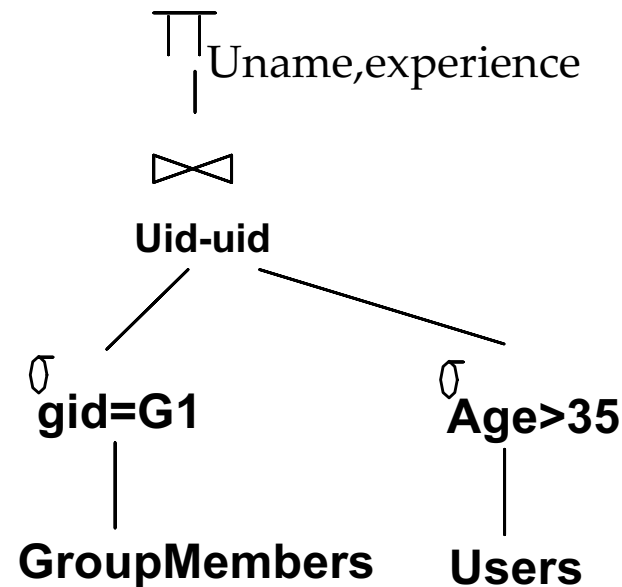
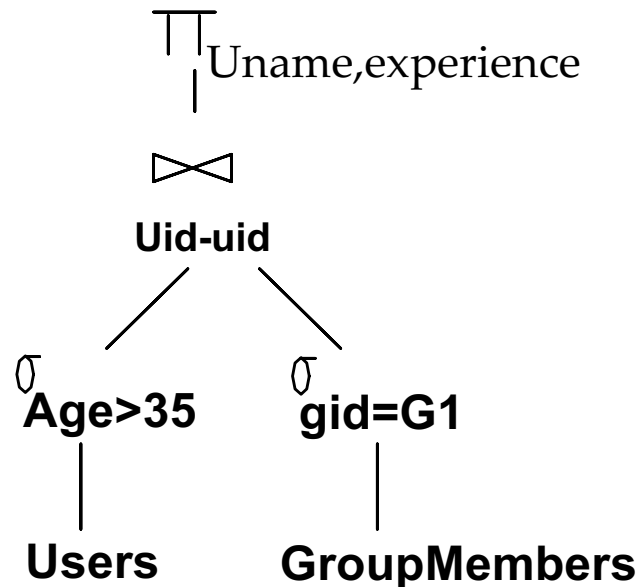
Around 50 buffer frames available

```

SELECT u.uname, u.experience
FROM   Users u, GroupMembers g
WHERE  u.uid = g.uid
AND    g.gid = 'G1'
AND    u.age > 35

```

Algebraic Optimization with selection push:



But do we really push all selections?

And do we use the indexes or not?

And what join type do we use?

Example

❑ Should we push the selections down or not?

❑ Do we use the index or not?

☆ *Users:*

- Non-clustered index on age not useful
- Scan to find qualifying tuples
 - 20,000 qualifying tuples
 - I/O Costs: 500 pages to read in and possibly 250 to write out if no pipelining to next operator possible
- Might consider doing selection sometime later

☆ *GroupMembers:*

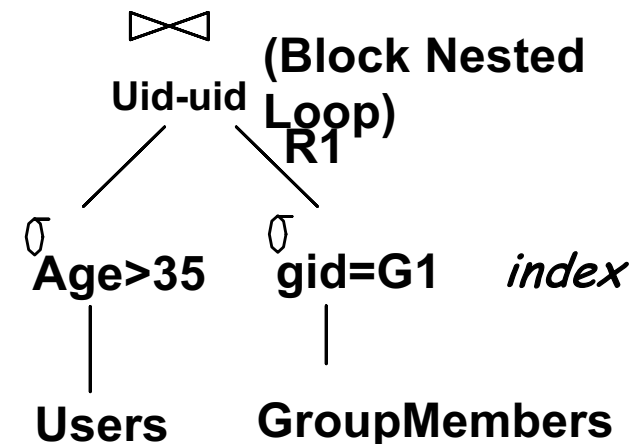
- index on *gid* matches *gid=g1*;
 - ▲ assume 500 different different groups and uniform distribution of 100000 records: 200 matching tuples
 - ▲ Worst I/O cost: 1-2 leaf page + 200 data pages
- Scan
 - ▲ Cost: 1000
- Output: 200 tuples = 2 pages (can stay in main-memory!)
- Very likely best to do selection first with the index

σ
gid=g1 *index*
|
Group Members

Example contd

What join implementation?

- Merge Sort join:
 - Sort 200 tuples of GroupMembers: main memory
 - User: If we do selection on age first
 - Sort 250 pages of Users > 35
 - Pass 0 of sort receives the 250 pages from selection in pipelines fashion
 - Pass 1 of sort pushes output into merge sort
 - Thus: 250 to write out Pass 0, 250 to read in for Pass 1 = 500
 - Total I/O costs selections and join
 - 1+200+ 500 + 250 + 250
- Users as outer:
 - Block Nested Loop join:
 - Precompute inner relation with selected group members on two pages in main memory
 - Pipeline output of age selection on User into join
 - Total I/O costs selections and join
 - 1+200+500
 - Index Nested Loop join:
 - not possible as
 - no index on uid of GroupMembers
 - already use index on gid of GroupMembers



Example contd

- Selected GroupMember as outer:

- Block Nested Loop:

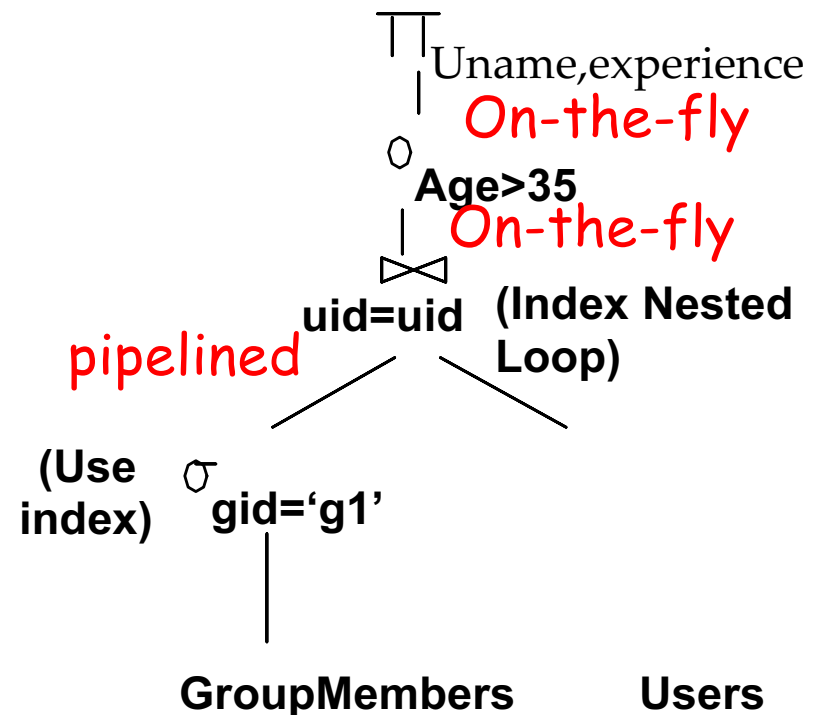
- I block of out relation (selected group members on two pages in main memory)
 - Read in users as inner, join and do selection on age on the fly
 - Total I/O cost for selections and joins:
 - I + 200 + 500

- Index Nested Loop:

- possible because of uid-index on Users:
 - But then we CANNOT do the selection on age before the join!!
 - for each qualifying GroupMember tuple get corresponding user through index
 - Then do selection on age on the fly after the join
 - I/O Cost for join only: $200 * (1 \text{ leaf} + 1 \text{ data-page}) = 400$
 - Total I/O cost for selection and joins
 - I + 200 + 400

- Index nested loop overall the best

- With age selection and projection on the fly right after a joining tuple is found



Nested Queries

- ❑ `Select S.sname`
`From Skaters S`
`Where S.sid IN (Select P.sid From Participates P`
`Where P.cid = 103)`
 - ☆ Execute subquery first; returns intermediate relation, with distinct sid
 - ☆ Join outer relation and intermediate relation with any join method
- ❑ `Select S.sname`
`From Skaters S`
`Where exists (Select * from Participates P`
`where P.cid = 103 and P.sid = S.sid)`
 - ☆ Have to execute inner query for each outer tuple; little optimization possible
- ❑ `Select S.sname`
`From Skaters S, Participates P`
`Where S.sid = P.sid AND P.cid = 103`
 - ☆ all optimizations possible
- ❑ smart rewriting system able to transform queries automatically

System Tuning

- ❑ If your application has some standard, well-known queries:
 - ★ Create appropriate indices to speed up these queries
- ❑ If your application has many many updates and inserts
 - ★ Be careful with creating indices
 - ★ Each INSERT will insert new value in each index of the table
 - ★ Updates change some indices
- ❑ SQL Explain explains how a query is executed internally

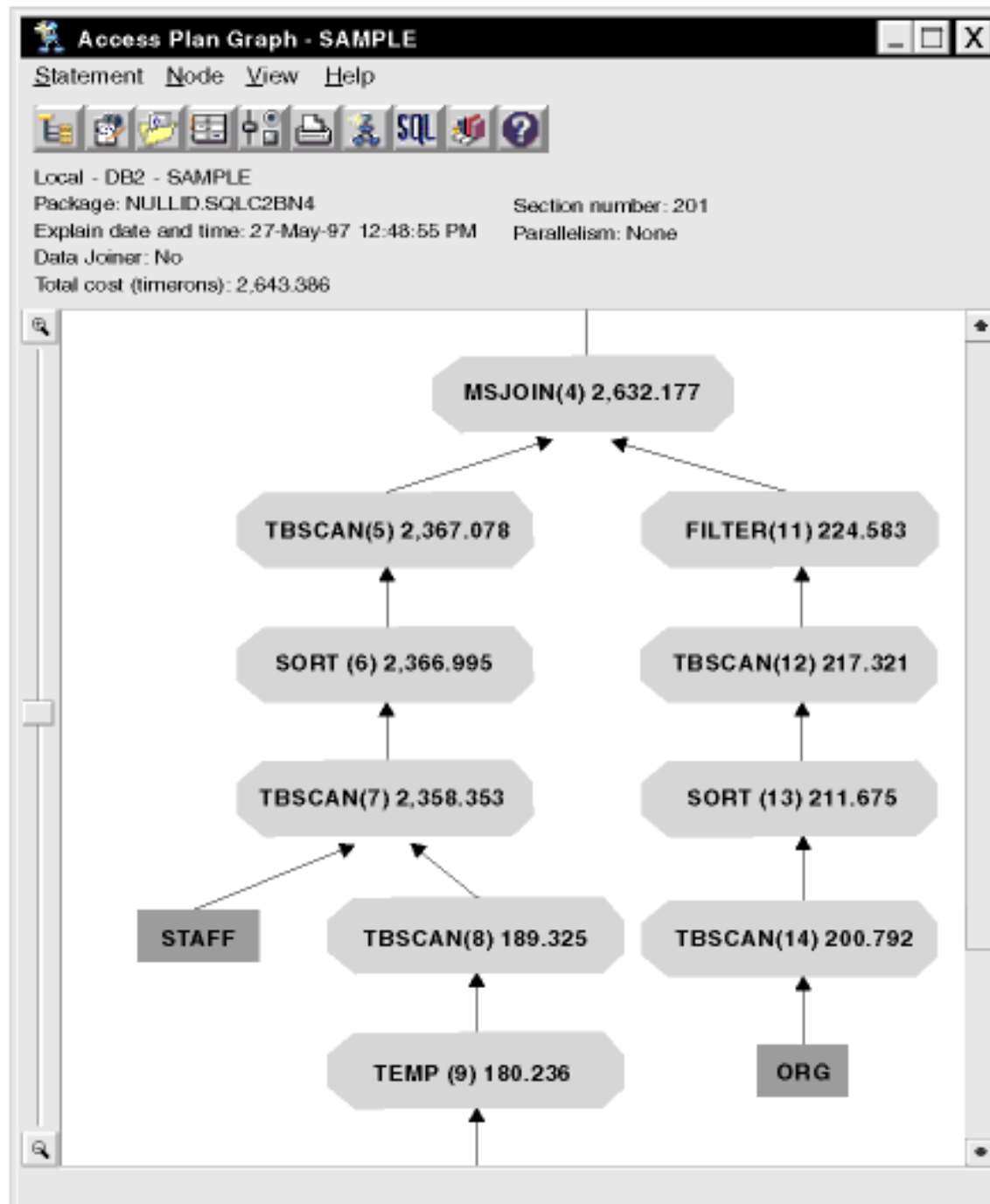
Statistics in DB2

- ☆ SYSCAT.TABLES and SYSSTAT.TABLES

- ☆ Number of pages per table, number of tuples, etc.

- ☆ SYSCAT.INDEXES and SYSSTAT.INDEXES

- Number of index leaf pages, index levels, degree of clustering, number of distinct values in first column, page fetch estimate for different buffer size, etc.



Operator details - MSJOIN[4]

Local - DB2 - SAMPLE

Level of details: ☐ Overview ☒ Full

Cumulative cost		
Total cost	2,632.177 timerons	
CPU cost	55,853.416 instructions	
I/O cost	15.921 I/Os	
First row cost	2,587.959 timerons	

Cumulative properties			
Tables	DENISEW.STAFF		
	DENISEW.ORG		

Input arguments			
Join predicates	Number 3	Selectivity 0.0024999999	Text (Q5.DEPTNUM=Q4.DEPT)

Save As...

Print...

Close

Help